

HousingChoice Platform

Technical System Architecture & Build Plan

Phase 1 planning deliverable — architecture, technology decisions, and AI-agent-driven build estimate

Prepared for HousingChoice · Founder: Samantha James · Atlanta, GA
June 11, 2026 · v2.12 · Itemized cost model for founder review

1. Executive Summary	3
2. Scope and Source Documents	3
3. Key Technology Decisions.....	4
4. System Architecture	6
4.1 Overview.....	6
4.2 The Critical Path: an Inbound Text	6
4.3 AWS Service Inventory	8
4.4 Application Structure	9
5. Data Model (DynamoDB).....	11
5.1 The Matching Engine vs. DynamoDB — the Honest Caveat	13
6. AI Conversation Layer (Phase 2).....	13
7. External Integrations	14
7.1 Texting & Voice — Twilio (and why not Quo)	14
7.2 The Rest.....	20
8. Infrastructure as Code & Delivery	22
8.1 Local Development.....	22
9. Logging & Observability	22
10. Security, Privacy & Compliance.....	24
11. Build Plan & Effort Estimate.....	25
11.1 Assumptions	25
11.2 Phase 0 — Foundations (1–1.5 weeks).....	25
11.3 Phase 1 — Feature Parity on the Platform (4–7 weeks)	25
11.4 Phase 2 — Matching Engine + AI Layer (4–7 weeks).....	26
11.5 Phase 3 — Automation Depth (4–6 weeks).....	27
11.6 Phase 4 — Scale, Invoicing + In-App Calling (3–6 weeks)	27
11.7 Summary & Costs	28
12. Risks & Mitigations.....	32
13. Open Questions	32
14. Parked for Later (Review Decisions)	33
15. Key References	33

1. Executive Summary

HousingChoice is a text-first tenant-placement engine for Section 8 voucher holders: it captures tenant needs through natural conversation, matches them only to units they fit and are likely to be approved for, and shepherds tenant and landlord through tours, applications, and the 30–45 day approval window to move-in, when the landlord is invoiced a placement fee. Today this runs on a hand-stitched MVP (Quo + n8n + Airtable/Supabase + Claude). This document is the blueprint for rebuilding it as a production platform on AWS, and estimates the effort to do so assuming AI coding agents perform most of the build.

The proposed system is a Node.js/Express modular monolith running in Docker on a single EC2 instance (t4g.small in a public subnet locked to CloudFront — review decisions: no WAF, no NAT gateway), with DynamoDB as the primary datastore, S3 for media, direct Google OAuth for team sign-in (no Cognito), EventBridge Scheduler for reminders, SES for email, Parameter Store for all config and credentials, and CloudWatch for an aggressive logging and observability posture. Everything is defined in Terraform with no console-managed resources; development, tests, deploys, and Terraform applies all run through npm scripts from the developer's machine — no CI/CD automation yet, by design. Claude powers the (Phase 2) conversation layer behind a provider adapter that switches between Amazon Bedrock, the new Claude Platform on AWS, and the direct Anthropic API with an environment variable.

The most consequential research finding: Quo (formerly OpenPhone), the current texting provider, cannot power this platform. Its public API sends to exactly one recipient and does not support MMS at all — confirmed against its live API specification — which rules out both programmatic group texts (tenant + landlord, a hard requirement of Track 6) and photo sharing (core to Track 3). The decision is Twilio — 1:1 texting via the Conversations API, and group communication via masked relay threads (review decision: phone numbers must never leak between tenants and landlords): each placement gets a dedicated pool number and the platform fans messages out with name prefixes. This sidesteps native group MMS entirely (its 2022–2024 availability saga is documented in §7.1); Telnix remains the documented fallback provider behind the messaging adapter. This also puts A2P 10DLC re-registration (roughly one to three weeks of carrier lead time) on the critical path and it should be started immediately. The decision (June 2026) is to retire Quo entirely: Twilio carries voice as well as texting — inbound calls bridge to team cell phones with a whisper announcement, outbound goes click-to-call with the business caller ID, and every call is recorded and transcribed into the database.

Estimated build effort, assuming AI agents write the large majority of the code under one technical lead's direction: **Phase 1 (feature parity — the team communicates manually through the platform, everything tracked) ≈ 4–7 weeks after a 1–1.5 week foundation stage; Phase 2 (matching engine + AI layer) ≈ 4–7 weeks; Phase 3 (automation depth) ≈ 4–6 weeks; Phase 4 (scale, invoicing, and the native call companion app) ≈ 3–6 weeks — roughly 16–27 weeks (~4 to 6.5 months) for the full roadmap.** The parity-first re-phasing (per founder review) front-loads a usable product: Quo retires and the team runs entirely on the platform at the end of Phase 1, roughly 5–8 weeks in; everything after improves a working system rather than delaying one. Itemized run cost during Phase 1 is roughly \$115–145/month (AWS is only ~\$25 of it — messaging volume is the swing factor); at full build, roughly \$210–290/month plus revenue-linked Stripe fees. §11.7 itemizes every line with quantities and rates for founder review.

2. Scope and Source Documents

This plan synthesizes three sources: the **Product Framework & Scope** (product definition, guiding principles, phasing), the **Master Process Flow SOP** (the eight operational tracks from landlord cold call to invoicing), and the **Product Spec & Feature Flow v2** (33 feature cards specifying, for every step, its trigger, actor, channel, data writes, build status across the two existing MVPs, and build

phase — it supersedes the earlier Technical Process Map). Where sources conflict, the v2 spec wins.

In scope for this document: the complete technical architecture, technology selections verified against live vendor documentation as of June 2026, the data model, development/deployment workflow, logging strategy, and a phased effort estimate. Out of scope: implementation itself, visual/UX design, pricing strategy, and migration mechanics for existing MVP data (flagged as an open question in §14).

Constraints set by you and honored throughout: AWS-hosted using as many AWS services as practical; Node.js with Express; everything containerized with Docker on an EC2 instance; CloudFront in front; full infrastructure-as-code with Terraform (zero drift); git source control; first-class local development; and maximal logging for cloud troubleshooting. Revised per founder review: lean-first infrastructure (no WAF, NAT gateway, Cognito, Secrets Manager, PITR, or CI/CD automation yet — each a clean later addition), npm-script-driven workflows, and parity-first phasing with the matching engine and AI layer deferred to Phase 2.

3. Key Technology Decisions

Each choice below was verified against live documentation in June 2026. Sunset checks that mattered: Amazon Pinpoint reaches end of support October 30, 2026 (its SMS successor, AWS End User Messaging, lacks group MMS and is not used); the AWS X-Ray SDK entered maintenance mode in February 2026 (OpenTelemetry is used instead); DynamoDB-based Terraform state locking is deprecated (native S3 locking is used); and Cal.com went closed-source in April 2026 (tour scheduling is built in-app).

Decision	Choice	Why
Runtime & framework	Node.js 22 LTS + Express	Your requirement; mature ecosystem; one language across app, jobs, and tooling.
Compute	Docker Compose on one EC2 t4g.small (Graviton/ARM; app + worker containers)	Your requirement; t4g.small per review (~\$12/mo). Images built for ARM64. Clean later path to ECS if needed — same images.
Edge	CloudFront + ACM TLS; EC2 in a public subnet locked to CloudFront	Per review: no WAF, no NAT gateway. The instance keeps a public IP for outbound API calls; inbound is restricted to CloudFront's managed prefix list plus a secret origin header the app verifies. Caching disabled on /api and /webhooks; static dashboard from S3.
Database	DynamoDB (on-demand, PITR on from day one — review reversal)	Your NoSQL requirement — schema-flexible documents tolerate heavy iteration. DynamoDB Local gives identical local dev. See §5 for the matching-engine caveat and scale path.
Geocoding	Amazon Location Service	v2 F3.1 requires geocoded addresses (subzones, transit distance, 'near my kids' school') for the normalization gate and matching. AWS-native, pay-per-request. Lands with the Phase 2 matching engine.
Texting	Twilio: Conversations API for 1:1; masked relay threads for groups (pool number per placement — review: no number leakage between	Quo's API cannot send MMS or to more than one recipient; AWS End User Messaging has no group MMS. Twilio's group texting is GA and purpose-built. See §7.1.

Decision	Choice	Why
	tenants and landlords); Telnyx is the documented fallback	
Voice	Twilio Programmable Voice + Conversational Intelligence (same number as SMS)	Full move off Quo; the number ports from Quo (review decision). Inbound rings the founder's cell for now; outbound via click-to-call or a dial-through access number; recordings + transcripts (~\$0.024/min) saved to the record — structured extraction is Phase 2. End state (Phase 4): a native companion app (iOS + Android) rings in-app with full caller context via CallKit/ConnectionService; the cell bridge stays as fallback. See §7.1.
AI	Claude behind an LLM adapter: Bedrock Claude Platform on AWS Anthropic API, switched by env var	Your requirement. Hedges the Bedrock access friction you have hit; Claude Platform on AWS (GA May 2026) offers the native API with IAM auth and AWS billing.
Email	Amazon SES	Welcome emails and document sends; the official Pinpoint-email successor; \$0.10 per 1,000.
E-signature	DocuSign eSignature API (Starter, \$50/mo, 40 envelopes)	Landlord brand trust; cheapest at current volume. Dropbox Sign if volume passes ~50/mo.
Lockbox tours	Igloohome API (\$2/active lock/mo)	Only major vendor with a public API; offline time-bound PINs fit the tour flow. Rently has no public API; 'Rentlee' does not exist.
Tour scheduling	Built in-app on the platform's own SMS layer	Calendly caps SMS reminders; Cal.com self-hosting is no longer production-safe. The domain is thin: slots, conflicts, reminders.
Invoicing	Stripe Invoicing, ACH-first	≈\$7 total on a \$2,000 ACH invoice vs ≈\$60 by card or ≈\$20 QuickBooks ACH.
IaC	Terraform ≥1.15, AWS provider v6, S3 state with native lockfile	Your requirement. S3 use_lockfile replaces the deprecated DynamoDB lock table. Applies run from the dev machine via npm scripts; drift is checked with a manual npm run plan (automation later).
Source control / delivery	GitHub for source; npm scripts for test, build, deploy (→ ECR → SSM restart)	Per review: nothing automated for now — npm run test, then npm run deploy from the dev machine. CI/CD added later when the team grows.
Observability	pino JSON logs → CloudWatch (awslogs driver), Logs Insights, metric alarms, OpenTelemetry traces → Application Signals	Your max-logging requirement; X-Ray SDK is end-of-life Feb 2027 so OTel from day one.
Team auth	Direct Google OAuth (Workspace-domain restricted)	Per review: no Cognito. ~50 lines with openid-client; sessions + founder/VA RBAC live in the app. Auth sits behind thin middleware, so a later swap to Cognito (e.g., landlord or housing-

Decision	Choice	Why
		authority logins, who won't have Workspace accounts) is days, not weeks — parked in §14.

4. System Architecture

4.1 Overview

A modular monolith: one Express application (API, webhooks, dashboard API) plus one worker container (matching runs, reminders, follow-up jobs) sharing a codebase and Docker image, orchestrated by Docker Compose on a single EC2 t4g.small in a public subnet whose security group admits only CloudFront (managed prefix list + a secret origin header the app verifies) and SSM — the public IP exists for outbound API calls, which eliminates the NAT gateway (review decision). CloudFront is the sole inbound entry point, routing /api/* and /webhooks/* (uncached) to the EC2 origin and serving the React dashboard and property media from S3. All state lives in managed services (DynamoDB, S3), so the instance itself is disposable and rebuildable from Terraform plus the latest image. One cross-cutting v2 requirement shapes the worker: the extraction pipeline — every text thread and call transcript is parsed so tenant, landlord, and unit fields populate from conversation, and nothing said by text or on a call is ever typed twice. (The extraction pipeline is Phase 2; in Phase 1, threads and transcripts are stored verbatim and fields are entered by the team.)

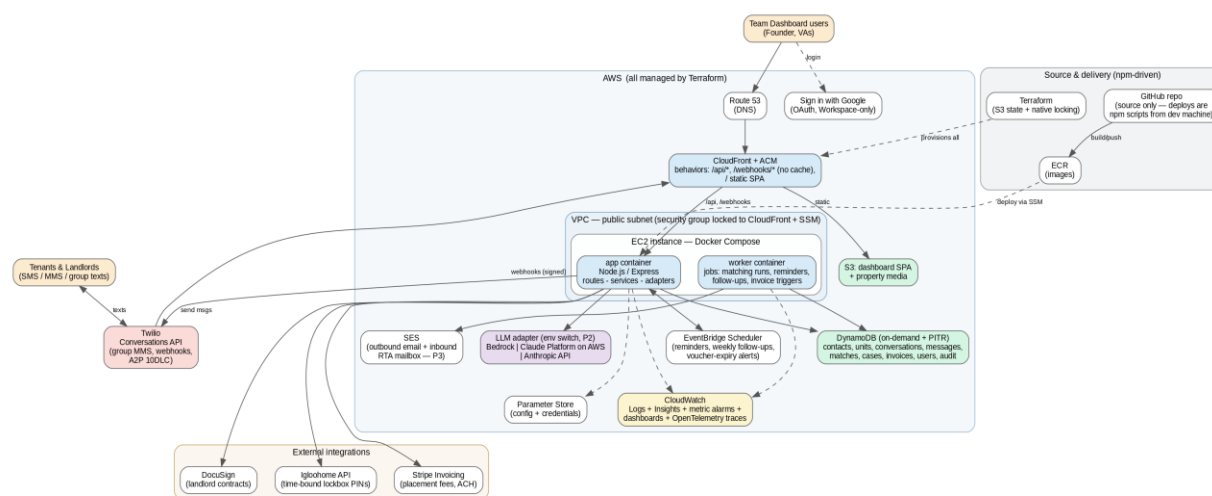


Figure 1 — Target architecture. Solid lines are runtime data flows; dashed lines are auth, configuration, deployment, and telemetry.

4.2 The Critical Path: an Inbound Text

Everything in this product hangs off one flow — a tenant or landlord sends a text:

1. Twilio receives the SMS/MMS and POSTs a signed webhook to CloudFront → /webhooks/twilio. The app verifies the HMAC signature (never source IP — CloudFront masks it), acks immediately, and enqueues processing.
2. The message is persisted to the messages table; any MMS media is copied from Twilio to S3. The conversation record is loaded or created and linked to the tenant/landlord by phone number.

3. Phase 1: the team sees the message instantly in the hub (SSE + PWA push) and replies manually — every message tracked with full audit. From Phase 2, the AI orchestrator assembles context — history, profile, matches, the property's stored processes — and calls Claude through the LLM adapter.
4. From Phase 2, per the conversation's ai_mode (two modes in the v2 spec): Auto sends the reply via Twilio; Manual stays silent while the AI keeps listening, extracting fields, and posting a suggested reply a teammate can edit or send with one tap. Hard guardrails (money, legal, fair-housing topics, distress, explicit request for a human, teammate already active) force a handoff regardless of mode. Every decision is logged with its reason (§10).
5. Side effects fan out: profile attribute updates extracted from the message, case stage transitions, reminder (re)scheduling via EventBridge, and live dashboard updates over SSE.

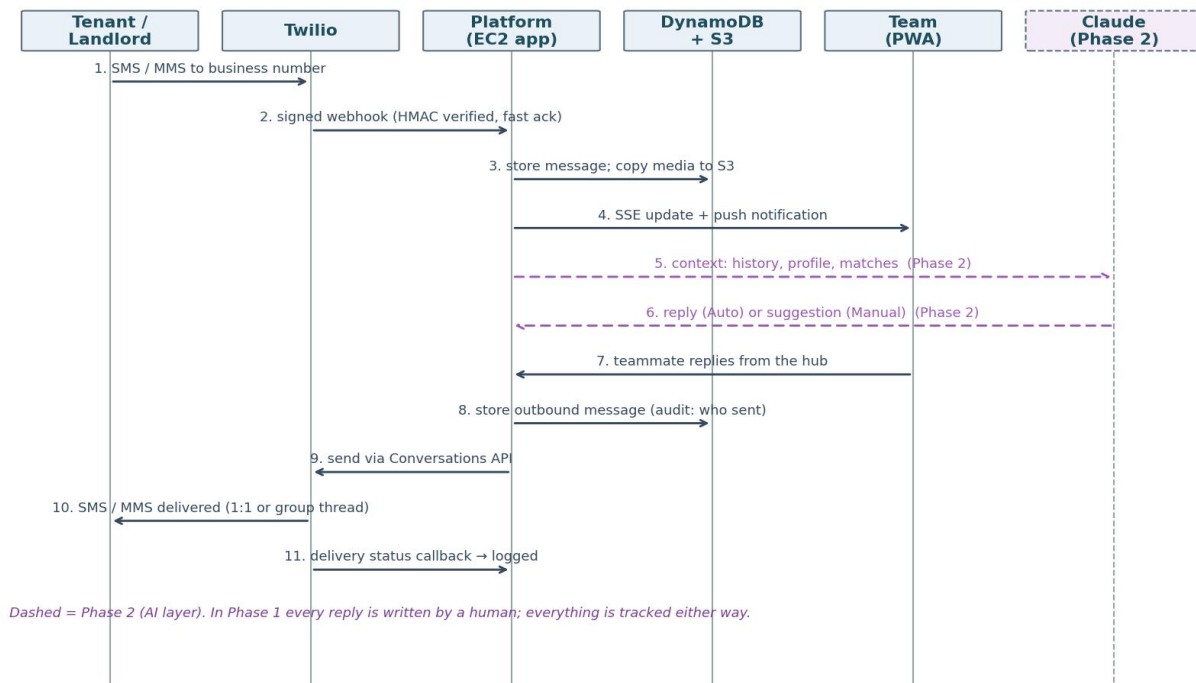


Figure 2 — Message lifecycle sequence (requested in review). Solid = Phase 1 (human replies); dashed = the Phase 2 AI layer slotting into the same flow.

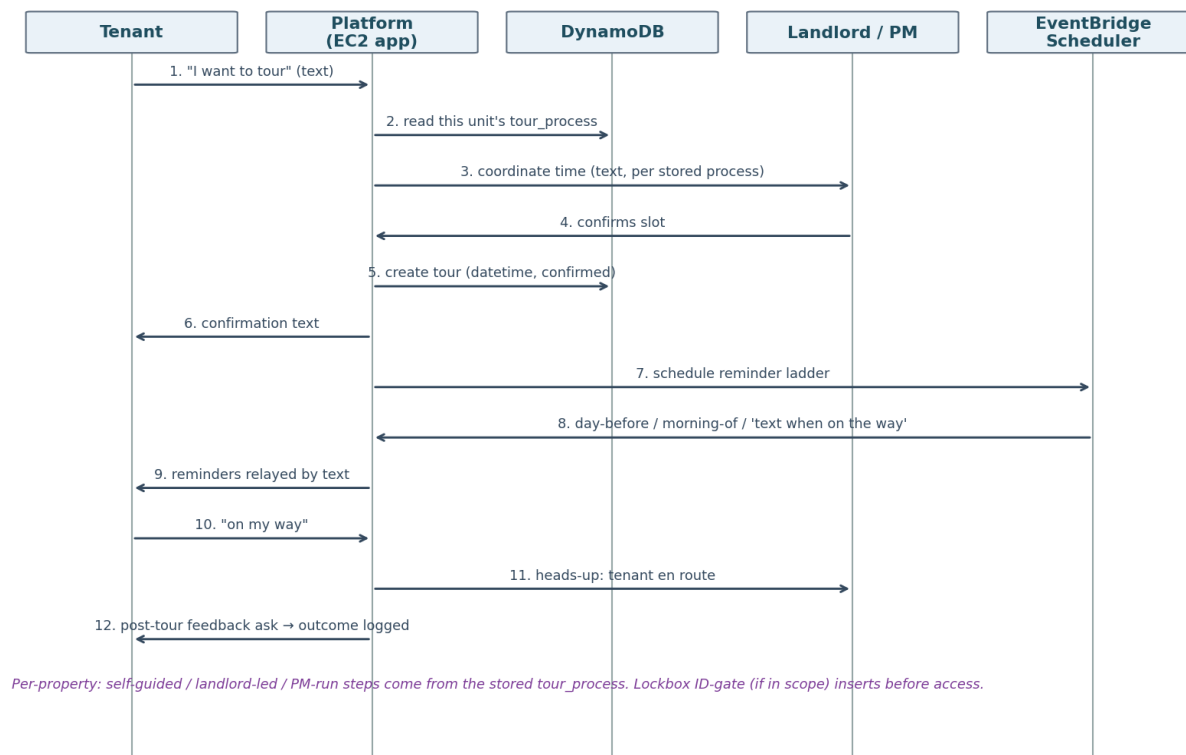


Figure 3 — Tour scheduling sequence (requested in review). The landlord/PM leg follows whatever that property's stored tour_process dictates.

4.3 AWS Service Inventory

Service	Role in the platform
EC2	Single t4g.small (Graviton) in a public subnet, security group locked to CloudFront + SSM; runs Docker Compose: app + worker. Auto-recovery alarm; stateless and disposable. Public IP handles outbound API calls — no NAT gateway (review).
CloudFront + ACM	TLS termination, static asset caching, uncached proxying of API/webhooks to the EC2 origin (prefix-list + secret-header lock). WAF dropped per review — clean to add back if abuse appears. ACM cert in us-east-1.
Route 53 (optional)	Only if DNS moves here — a CNAME from wherever housingchoice.com is hosted today works fine (open question §13).
DynamoDB	Primary datastore, on-demand billing, point-in-time recovery enabled from day one (review reversal — message history is the business record; ~\$0.20/GB-mo is noise), streams for event-driven side effects. DynamoDB Local in dev.
S3	Property photos/videos and MMS media (served via CloudFront), dashboard SPA bundle, Terraform state (with native locking), log/CI artifacts.
ECR	Docker image registry; ARM64 images built and pushed by npm run deploy from the dev machine, promoted dev → prod by tag.

Service	Role in the platform
Google OAuth (no AWS auth service)	Team sign-in via Sign in with Google, restricted to the Workspace domain; sessions and founder/VA RBAC live in the app (DynamoDB). Pluggable middleware keeps a later Cognito swap cheap (review).
EventBridge Scheduler	One-off and recurring schedules: tour reminders, 'text me when on the way' nudges, weekly Track-6 follow-ups, voucher-expiration alerts, RTA 48-hour checks. Cleanup is structural: fired one-off schedules do not delete themselves by default — they linger indefinitely as dead resources — so the enqueue wrapper sets ActionAfterCompletion: DELETE on every one-off schedule, and canceling a job (tour rescheduled, case lost) deletes its schedule explicitly. A weekly sweep logs any orphans as a regression signal.
SES	Outbound welcome/onboarding emails; inbound mailbox (SES receiving → S3 → worker) for hello@ RTA submissions, with AI receipt confirmation (v2 F5.4).
Amazon Location Service	Geocoding for the data-normalization gate (v2 F3.1): unit addresses, subzones, and tenant area/transit preferences resolved to coordinates for commute matching. Phase 2, with the matching engine.
Parameter Store (SSM)	All credentials as SecureString parameters, all config as parameters — standard tier is free. Secrets Manager dropped per review (no rotation requirement yet).
CloudWatch	Log groups per container, Logs Insights, metric filters → alarms → SNS, dashboards, OpenTelemetry traces via Application Signals.
Bedrock (or Claude Platform on AWS)	Claude inference inside the AWS boundary when access is granted; direct Anthropic API otherwise — switched by env var.
SSM (Systems Manager)	Session Manager for shell access (no SSH keys/bastion) and Run Command for zero-downtime-ish deploys.
SNS	Alarm fan-out to email/SMS for the on-call human.

4.4 Application Structure

One repository, three top-level deployables, with provider adapters isolating every external dependency the platform may need to swap:

```

housing-choice/
├── app/                                # Node.js/Express + worker (one image)
│   ├── src/routes/                    # api/, webhooks/, health
│   ├── src/services/                 # conversations, intake, matching, cases
│   │                                     # (tour → move-in), invoicing, boards
│   ├── src/ai/                       # orchestrator, prompts, guardrails, evals
│   ├── src/adapters/                 # messaging/ (twilio|telnyx), llm/ (bedrock|
│   │                                     #   claude-aws|anthropic), esign/, locks/, billing/
│   ├── src/repos/                    # DynamoDB repositories (one per table)
│   ├── src/jobs/                     # worker entrypoint + scheduled job handlers
│   └── src/lib/                       # logger, config, errors, telemetry
├── dashboard/                        # React SPA (inbox, boards, profiles) – PWA
├── infra/                            # Terraform: modules/ + envs/{dev,prod}
├── docker-compose.yml                # production composition on EC2 (local dev is npm-first)
└── scripts/                          # npm run dev | test | deploy | plan | apply – no CI/CD yet

```

The dashboard is the team's working surface: a shared inbox (conversation list, thread view, assignment — joined in Phase 2 by the per-conversation `ai_mode` toggle, one-tap AI suggested replies, and escalation queue), the pipeline board (new → matched → touring → applied → in approval → placed), tenant/unit/landlord profile editors including each property's free-form `tour_process` and `application_process`, and an operations view of reminders and alerts. Live updates arrive over server-sent events through CloudFront. The dashboard ships as an installable, mobile-first PWA with push notifications — on a phone it is the team's texting and calling surface, so no native app (and no Quo) is needed. Why React (review question): the hub is a stateful, live-updating SPA — inbox over SSE, PWA install, push notifications — which is the shape of app React's ecosystem serves best, and the one AI coding agents are most fluent in; Vue or Svelte would also work. It's a default, not a dogma.

Memory-pressure guideline (binding for AI agents). All file movement — MMS media to S3, call recordings, RTA email attachments, uploads — must use Node streams (`stream.Readable` / `stream.Writable`, piped via `stream.pipeline`) end to end; `fs.readFileSync` and whole-file in-memory buffers are prohibited on media paths. On a `t4g.small` (2 GB), a handful of concurrent 5 MB MMS handled as buffers is the difference between steady state and an OOM kill; streams keep memory flat regardless of media size. Enforced by a lint rule (`no-restricted-syntax` on `readFileSync` under `src/`) plus code review.

Express middleware order (locked at review). (0) Correlation-ID injection — position zero, so every request including rejected ones logs under a `requestId`. (1) Light logger: method, path, raw socket IP plus X-Forwarded-For (XFF untrusted until validation — behind CloudFront the socket IP is an edge node), and redacted headers — the secret origin header and any credentials are never written to logs. (2) CloudFront secret-origin validator: mismatch fires a WARN/ERROR with the offender IP and path under the already-injected correlation ID and returns 403 immediately — no body is ever parsed for unauthenticated traffic, which doubles as cheap body-based-DoS protection. `/health` is exempt (deploy health checks arrive via localhost without the CloudFront header). (3) Body parsers — `express.json` with raw-body capture, required downstream for Twilio HMAC signature verification — and deep loggers; then routes.

5. Data Model (DynamoDB)

Clean-sheet redesign. Rather than mirroring the v2 spec's screen-by-screen tables, this model is organized around how the data actually behaves: who people are, what was said, what the engine computes, how a deal progresses, and what happened. Three consolidations do most of the work. First, **contacts** unifies tenants, landlords, PMs, and landlord team members — resolving an inbound phone number to a person is the hottest lookup in the system and now takes one query against one table, while type-specific data (voucher, prescreen, preferences for tenants; lead status, decline reasons, approval criteria for landlords) are simply document attributes. Second, **calls are messages**: a call lives in the messages timeline with its recording key and transcript, so the hub renders one interleaved text-and-call history per conversation with no joins. Third, **the journey is one record**: what v2 modeled as five tables (matches → tours → applications → rta_cases → placements) is, from tour interest onward, a single cases item moving through a stage ladder — because it is one deal. The pipeline board becomes one query, and every clock in the business (voucher expiration, the RTA 48-hour window, stuck-case alerts, tour reminders) hangs off one sparse byNextDeadline index that answers 'what needs attention right now?' in a single query. Matches stays separate from cases deliberately: engine output is volatile and bulk-regenerated, while a case is precious and hand-touched — mixing them would pollute streams, metrics, and audit. Items remain flexible documents (only keys and GSI attributes are contractual), so schema churn during the build needs no migrations. On-demand billing; streams where noted; point-in-time recovery on from day one (review reversal).

Table	Keys	GSIs	Holds
contacts	PK contactId	byPhone; byTypeStatus; byHousingAuthority (tenants)	Every external person, typed tenant landlord pm team_member. Shared: phones (auto-saved as 'First Last - N Bed'), emails, consent/opt-in, notes. Tenant section: voucher_size + voucher_program (exactly one authority at a time — HUD-verified), rta_in_hand + rta_expiration_date, caseworker, lease_end_date, prescreen answers, preferences_notes. Landlord section: lead_status with decline_reason taxonomy, contract_status, registered flag, comms preference, team links, approval_criteria, authorities served.
units	PK unitId	byLandlord; byStatus; byJurisdiction	Geocoded address, primary HCV jurisdiction + accepted programs list (§13 question pending), physical facts, min/max rent vs payment standard, deposit/LIF, utilities, accessibility, pets, priority, media + listing_link flyer, and the never-standardized per-property tour_process and application_process documents.
conversations	PK conversationId	byParticipantPhone; byLastActivity	Thread header + inbox index: Twilio binding (1:1 Conversation SID, or relay pool number + members[] for group threads — §7.1), participants → contactIds, type (tenant 1:1 landlord 1:1 relay group), ai_mode (auto manual, Phase 2), assignment, denormalized last-activity/preview. Holds the state no message contains.

Table	Keys	GSIs	Holds
messages	PK conversationId, SK ts#msgId	—	The append-only log, texts and calls interleaved: type (sms mms call), body or recording S3 key + transcript, direction, author (tenant landlord teammate ai), provider IDs, delivery status, AI decision metadata (Phase 2). Stream feeds side effects.
matches	PK tenantId, SK unitId	byUnit	Volatile engine-output cache (Phase 2; Phase 1 logs broadcast shares here): fit_score + approval_likelihood with reasons, rank, status (surfaced → shared → interested/declined), portability_required flag with friction-discounted rank (\$5 scoping), share feedback. Bulk-regenerated on any data change; stale rows TTL away.
cases	PK caseId	byTenant; byUnit; byStage; byTourDate (sparse); byNextDeadline (sparse)	One deal, tour-interest → move-in: stage ladder (interested → [porting, when MATCH_WITH_PORTABILITY_REQUIRED] → touring → applied → rta_submitted → inspection → rent_determined → lease → moved_in lost), current tour + tour history, four-rung application ladder, RTA/approval data (inspection, rent_determined + tenant_portion, LIF, denial letters), group_thread → conversationId, lease/move-in dates, and next_deadline_at/type powering every alert clock.
invoices	PK invoiceId	byLandlord; byStatus	Financial records kept separate from cases for clean accounting: auto-created when rent_determined lands (Phase 4), amount = one month's determined rent (\$1,400–\$3,500), due at move-in, Stripe ID, sent_at → paid_at.
users	PK userId	byEmail	The team: Google identity (sub + email), role (founder_admin va), permission scopes. New in the clean-sheet model — RBAC needs a home once Cognito is out.
audit_events	PK entityKey, SK ts	byActor	Append-only audit: every AI decision, handoff, mode change, profile edit, send — the troubleshooting and compliance trail.

Keys and housing-authority scoping (review items). Why so few sort keys: most tables hold one item per entity, keyed by ID alone — sort keys earn their keep inside item collections (messages: PK conversationId + SK timestamp), while alternate lookups (phone → tenant) are GSIs; adding SKs elsewhere would buy nothing. On housing-authority scoping (review), verification against HUD and Georgia DCA sources confirms half the rule: a tenant's voucher is administered by exactly one authority at a time (it can change, but only through HUD's formal portability process), so tenant contacts carry a single authority field and a byHousingAuthority GSI — enforced at the validation gate. The unit side is not strictly one-to-one: a unit's address sits in one primary HCV jurisdiction (in Georgia, DCA covers 149 counties; Atlanta, Fulton, DeKalb and ~7 others run their own PHAs), but

portability lets a tenant from another authority lease the same unit (the receiving PHA bills or absorbs the voucher), and program vouchers such as GHV or Step Up overlay geography. Units therefore store a primary jurisdiction plus an accepted-programs list, and the engine treats jurisdiction as a cost rather than a wall: a cross-jurisdiction pairing is not rejected but emitted as `MATCH_WITH_PORTABILITY_REQUIRED` — provided the tenant passes a `port_eligible` check (HUD's first-year residency rule plus the initial authority's policy) — scored with a founder-tunable friction discount (porting adds ~3–6 weeks against the voucher clock), evaluated against the receiving authority's payment standard rather than the tenant's current one, and gated by a per-market config flag that controls whether portability matches surface at all. Accepting such a match alerts the team on day one to start HUD porting paperwork and opens a porting branch on the case's stage ladder, instead of the blocker surfacing weeks later at application. The remaining founder question in §13 narrows to appetite: surface ports from day one, or config-off until Atlanta is saturated?

5.1 The Matching Engine vs. DynamoDB — the Honest Caveat

Matching cross-references every active tenant against every active unit on many attributes at once (voucher size, rent vs payment standard, location, stairs, pets, then landlord criteria). That is ad-hoc multi-attribute querying, which DynamoDB does not index well. The pragmatic answer at your scale: the worker loads active units and active tenant contacts (hundreds to low thousands of small items — single-digit MB), runs matching in memory in Node, and writes scored results to the matches table. This is fast, costs pennies on on-demand reads, and keeps matching logic in plain testable code rather than query gymnastics. The documented upgrade path when a market exceeds roughly tens of thousands of active records: precomputed match-candidate indexes updated from streams, or an OpenSearch index fed by DynamoDB streams. No re-architecture required — the engine is a pure function behind a repository interface. With matching scoped to serviceable authorities (§5), each run's working set shrinks further still.

6. AI Conversation Layer (Phase 2)

Per founder review this entire layer is deferred to Phase 2: Phase 1 ships the hub with humans doing all the talking and everything tracked. The design below is unchanged and ready for when it switches on. The orchestrator implements the AI ↔ human model exactly as the v2 spec defines it. Per-conversation `ai_mode` is data, not code — a field a teammate flips in the inbox. Two modes (the v2 spec drops the Product Framework's earlier Draft mode): **Auto** — the assistant replies on its own, for safe routine message types only, with autonomy earned per message-type as trust builds; **Manual** — the assistant stays silent but keeps listening, extracting fields, and updating records in the background, and posts a suggested reply a teammate can edit or send with one tap (preserving Draft's benefit without a third mode — flagged for confirmation in §13). On every inbound message the orchestrator runs: context assembly → guardrail pre-screen → Claude call (with tool access to profiles, matches, and per-property processes) → action (send / suggest + stay silent) → full decision logging.

Hard handoff triggers (deterministic checks plus model self-assessment, any one forces the assistant silent — suggest-only — regardless of mode): upset or emotional tenant; anything touching money, legal questions, or disputes; sensitive or fair-housing-adjacent topics; low model confidence; an explicit request for a person; a teammate already active in the thread. Guardrails live in code and prompts under version control, with a regression suite of golden conversations (including red-team fair-housing probes) run in CI before any prompt or model change ships.

Provider switch. A thin LLM adapter exposes one interface; `LLM_PROVIDER=bedrock | claude_aws | anthropic` selects the backend and model ID at boot, per environment. Bedrock keeps inference inside the AWS boundary (note: Anthropic models on Bedrock still require a one-time use-case access request, which has been your friction point); Claude Platform on AWS (GA May 2026) offers

the native Anthropic API with IAM auth and AWS billing and is worth evaluating as the best of both; the direct Anthropic API is the day-one fallback that unblocks development immediately. The adapter logs provider, model, latency, and token usage on every call so cost and behavior are comparable across backends.

7. External Integrations

7.1 Texting & Voice — Twilio (and why not Quo)

Research verified against live vendor specs in June 2026: Quo's API sends to exactly one recipient (to: `maxItems 1`) and its own docs state MMS is not supported in the current API version — so the platform could neither run group texts nor send property photos programmatically, despite the Quo app itself supporting both. Dialpad's API allows group sends but is a per-seat UCaaS product whose webhooks redact message content by default. AWS End User Messaging (Pinpoint's SMS successor) has no group-thread concept at all. Twilio's Conversations API has group MMS as a GA, purpose-built feature: a conversation object holds up to 10 participants, every member sees a normal native group thread, webhooks fire per message, and the object model (conversation → participants → messages) maps one-to-one onto the conversation hub. Per review there is no fallback provider — Twilio only. (The thin messaging adapter remains as cheap insurance, but no second integration is built.)

Costs and constraints: ≈\$0.008/SMS segment, ≈\$0.022/MMS sent (group messages fan out per recipient), \$1.15/mo per number, A2P 10DLC ≈\$20–35 one-time + \$1.50–10/mo. Group-MMS threads have carrier-fixed membership — relevant only to the native-group alternative documented below; the chosen relay design has no carrier group thread at all (see Group threads below). Twilio Advanced Opt-Out handles STOP/HELP automatically for TCPA compliance. **Critical path: A2P 10DLC brand and campaign registration takes days to ~3 weeks and gates all production texting — file it the week the Twilio account is opened.** Review decision: port the existing Quo number. Sequencing: build on a temporary Twilio number, complete A2P registration, rehearse, then port in a scheduled cutover — the number keeps working on Quo until the port lands (typically 1–2 weeks). Review question answered: A2P registration does not carry over from Quo — brand and campaign must be re-filed through Twilio (needed: EIN, legal name and address, website, opt-in description, sample messages); since it was already done once for Quo, every answer is known.

Webhook echo chambers — a build-time guard, noted now. Conversations' `onMessageAdded` webhook fires for every message in a thread — including messages our own backend just posted. A naive handler ('save it, run side effects') therefore processes our outbound replies as new inbound traffic: the AI answers its own answer, recursively — spam and a bill spike within seconds. Defenses, all mandatory in the webhook middleware: (1) author check first — if the message author is our identity/projected address, it is an echo: acknowledge and stop, no side-effect pipeline; (2) idempotency by `MessageSid` — a conditional write dedupes both echoes and Twilio's own redeliveries (outbound messages are persisted at send time by the send wrapper, so the webhook copy is a no-op); (3) the orchestrator never triggers on messages authored by 'ai' or a teammate; (4) a per-conversation circuit breaker caps automated outbound per minute — which also defuses the out-of-band variant where our assistant and someone else's autoresponder (a PM's after-hours bot, a carrier bounce) reply to each other forever; trips alarm and flip the conversation to Manual. The golden-conversation test suite includes an echo-loop scenario so regressions fail CI before they fail production.

Broadcast throttling — A2P throughput is a budget, not an assumption. A2P 10DLC long codes carry strict carrier-enforced throughput tied to the brand's registration tier: AT&T allocates per-second rates by campaign class (a standard low-tier long code may be limited to roughly 1–5 messages per second), T-Mobile additionally enforces per-brand daily message caps, and MMS typically rates lower than SMS — with multi-segment messages each counting against the budget.

Exceeding the allocation gets messages queued, erred (429 / Twilio 30022), or quietly carrier-filtered — poison for deliverability reputation. So no code path ever calls 'send' in a loop: every broadcast (Share Properties to a filtered tenant list, reminder fan-outs, group relays) flows through the worker's outbound queue with a token-bucket throttle set below the registered tier's allocation (configurable; conservative default ~1 msg/sec at launch), jittered pacing, and exponential backoff with retry on throttling errors. Twilio's Messaging Service queue provides a second buffer, but the app-level throttle keeps the queue shallow and the daily T-Mobile cap respected — a 100-tenant property blast simply trickles out over ~2 minutes, which the founder's use case never notices. Queue depth and throttling-error rate feed the send-failure alarm, and the brand tier's actual allocation (assigned at A2P registration) is recorded in Parameter Store as the throttle's source of truth.

Undeliverable messages — closing the loop. Twilio reports every outbound message's journey via status callbacks (sent → delivered / undelivered / failed) with a specific error code, and Conversations provides per-recipient delivery receipts in group threads. The callbacks update the message item and SSE-push to the hub, where each message shows its state — queued → sent → delivered, or a failure badge with the human-readable reason — with per-recipient chips on group messages and retry / call-instead actions inline. Handling is error-class-aware: transient failures (30003, handset unreachable) get a scheduled retry with backoff; permanent ones flag the contact rather than the message (30005 invalid number, 30006 landline → sms_unreachable, prompt voice instead) and never retry; 30007 carrier-filtering feeds the deliverability alarm and is never retried (re-sending filtered content compounds reputation damage); 21610 confirms the opt-out suppression list. A failed send on an active case also drops an item into the escalation queue so a human calls — the message failing must not mean the communication fails, especially against voucher and RTA deadlines. Caveat encoded in the UI: carrier delivery receipts are best-effort, so 'sent' and 'delivered' are distinct states and a message resting at 'sent' is not treated as an error.

Group threads — masked relay design (review decision). Requirement update: phone numbers must never be exposed between tenants and landlords — which rules out native group MMS, since carrier group threads show every member's number to every other member. The plan is relay threads. Each placement gets a dedicated pool number (~\$1.15/mo, recycled at case close after a quarantine period); every member texts that number in a normal private 1:1 thread; the group exists only in the database (conversations.members[]); any inbound message stores once and fans out to the other members prefixed with the sender's name. What this buys: membership is truly mutable (add or remove freely — nothing churns on anyone's phone), no member cap, full number masking both directions, tenants and landlords cannot route around the platform, and — decisively — no dependency on any group-MMS product: relay needs only plain message sends, which ends the Twilio group-texting availability question for good. Costs roughly double per group message (each inbound triggers N-1 relays — pennies at three members) and relays run through the same broadcast throttle. The one real loss is the shared-room feel of a native group text; it is mitigated by intro messages that name everyone connected, and Figure 5 shows what both sides actually experience.

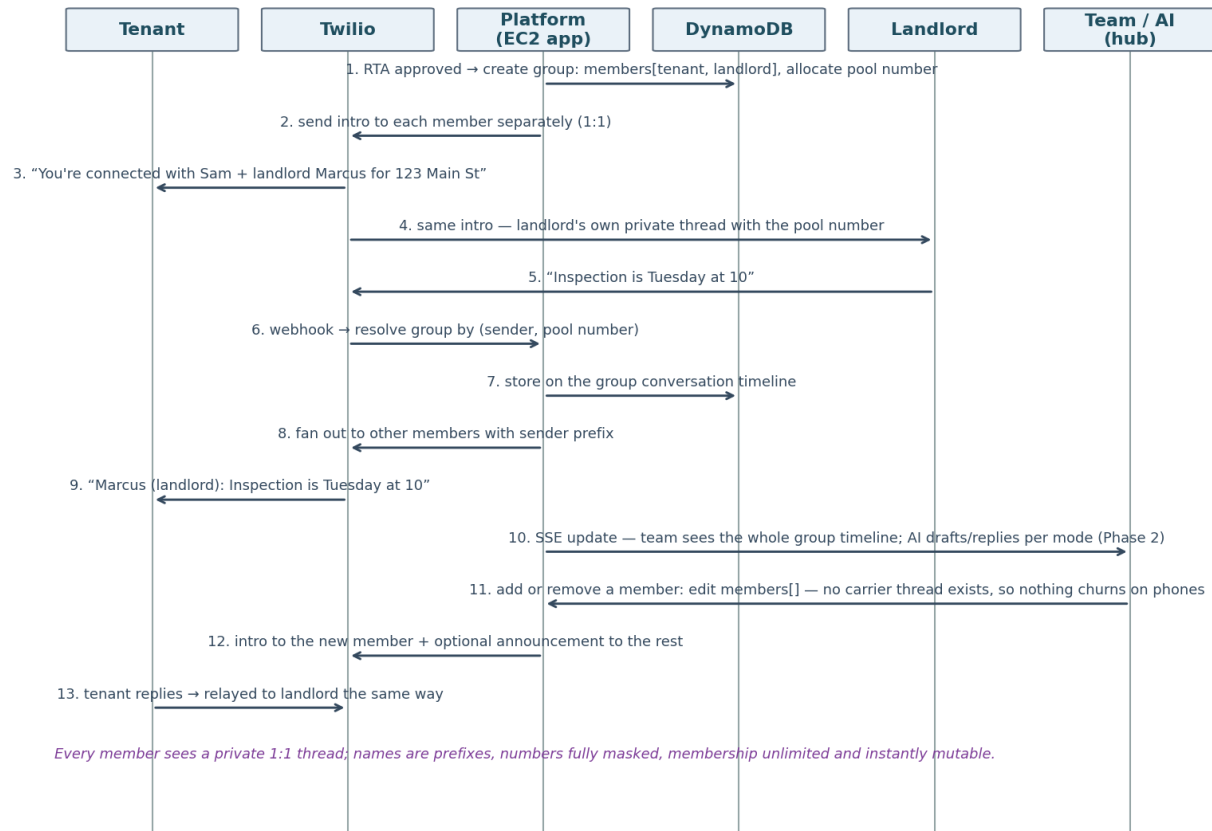
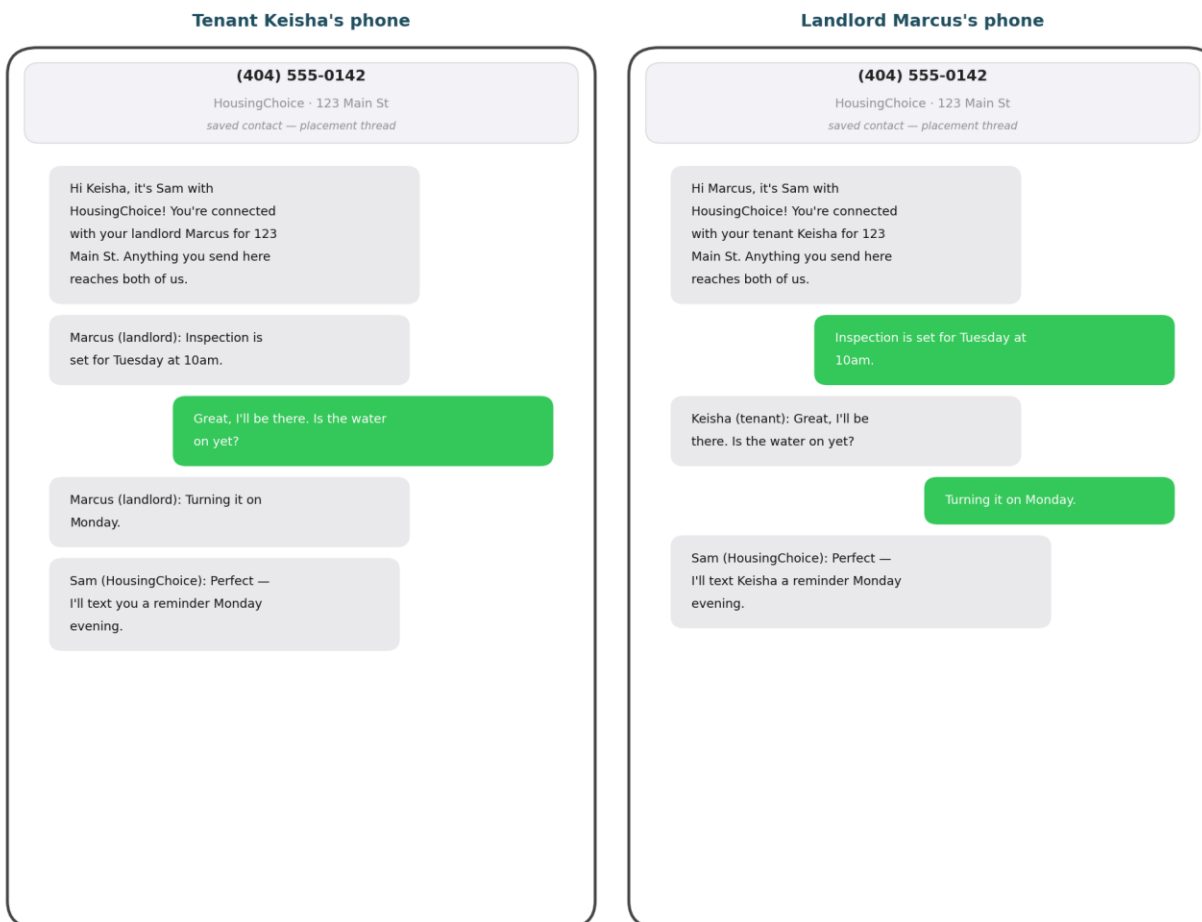


Figure 4 — Relayed group thread: the group-messaging design. The group lives in the database, not at the carrier; each member sees a private 1:1 thread with the placement's pool number.



Same conversation, two phones: each party texts the placement's pool number in a normal 1:1 thread; names arrive as prefixes; no personal number ever crosses sides.

Figure 5 — The same relayed conversation on both phones. Each party texts the placement's pool number in a normal 1:1 thread; names arrive as prefixes; no personal number ever crosses sides.

Alternative — native group MMS (documented, not built). Twilio's Conversations group texting is GA since October 2024 (a March 2022 changelog briefly gated it to existing accounts and was never annotated — resolved during research), creating groups of up to 10 via the Conversation with Participants API; carrier threads have immutable membership, so add/remove requires thread swaps. It delivers the shared-room psychology of a real group text but exposes every member's number to every other member — incompatible with the masking requirement, so it remains documented only in case that requirement is ever relaxed.

Voice — full Quo retirement. The same Twilio number carries calls. Inbound: a TwiML flow rings the founder's cell (review: founder only for now) with a whisper announcing the caller and context — and the whisper doubles as a human-answer gate: the bridge completes only on a keypress ('press 1 to accept'), which carrier voicemail cannot perform. This matters because if her phone is off, in a dead zone, or on Do Not Disturb, her carrier answers instantly with personal voicemail and Twilio would otherwise see an 'answered' leg — connecting the tenant to a personal Verizon greeting while our system logs a successful call. The keypress gate detects the non-human answer deterministically (no AMD guesswork or per-call fee), abandons that leg, and falls through to the platform's own voicemail: business greeting, recorded, transcribed, logged on the conversation timeline, PWA-notified. Genuinely unanswered calls take the same path, and the same gate protects the outbound click-to-call leg and makes future simultaneous ring safe (first human keypress wins; no one's carrier voicemail can capture the call). Known residual nuisance: each business call missed

while her phone is off deposits a short whisper clip (~10–15s of 'press 1 to accept' then silence) in her personal voicemail — mitigated by a terse whisper and a short keypress window (~6–8s, no repeat), and reduced further once the Phase 4 app makes the cell bridge a fallback. Optional escalations if it grates, each with real cost: answering-machine detection hangs up on detected machines before the whisper but adds 2–4s answer delay for humans too (\$0.0075/call, probabilistic); or disable her personal carrier voicemail entirely (eliminates collisions; personal callers lose voicemail). Ship with terse-whisper + short window and set expectations. Caller-ID reality (review question): her phone shows the business number calling — that is what marks it as a business call; the tenant's own number cannot appear in native caller ID, so context comes from the whisper plus the PWA push that fires simultaneously with the full contact record. (Passing the real caller number through is possible, but then business calls are indistinguishable from personal ones — default is business number + whisper; see §13.) Outbound: click-to-call from the dashboard — Twilio rings the teammate's cell first, then bridges to the landlord/tenant with the business number as caller ID, so personal numbers are never exposed. Per review, a dial-through access number is included too: dial a dedicated Twilio number from a registered cell, key in (or speak) the destination, and the call bridges out with business caller ID — no dashboard needed. Every leg is recorded (\$0.0025/min) and transcribed via Conversational Intelligence (\$0.024/min batch); transcripts are saved verbatim to the contact's record (review: just save the transcript for now — Claude's structured field extraction from transcripts is Phase 2). Rates: \$0.014/min outbound, \$0.0085/min inbound; at ~500–1,000 call minutes/month voice adds roughly \$20–45/month. A browser softphone in the dashboard (Voice JS SDK, \$0.004/min per leg) lands in Phase 3 for desk work. The mobile end state (review decision, Phase 4 — pull forward if the whisper experience grates; the Phase 3 softphone builds the same token/registration plumbing): a small native companion app — React Native with Twilio's official Voice React Native SDK, one codebase for iOS and Android — that rings like a real call via CallKit/ConnectionService. A VoIP push wakes it from a locked screen, the OS-level incoming-call UI carries the caller's name, the in-app screen shows the full contact card and case context, and answered calls run over data with the identical server-side recording → transcription → timeline tail (Figures 6–7 change only at the ring step). Routing ladder: ring the app for a few seconds → fall back to the PSTN cell bridge → voicemail, so a dead battery or no-signal moment never loses a landlord call. Until the app ships, bridged calls present the business number + whisper; a PWA alone cannot deliver native call UX (CallKit/ConnectionService are native-only), which is why this is an app and not a web feature.

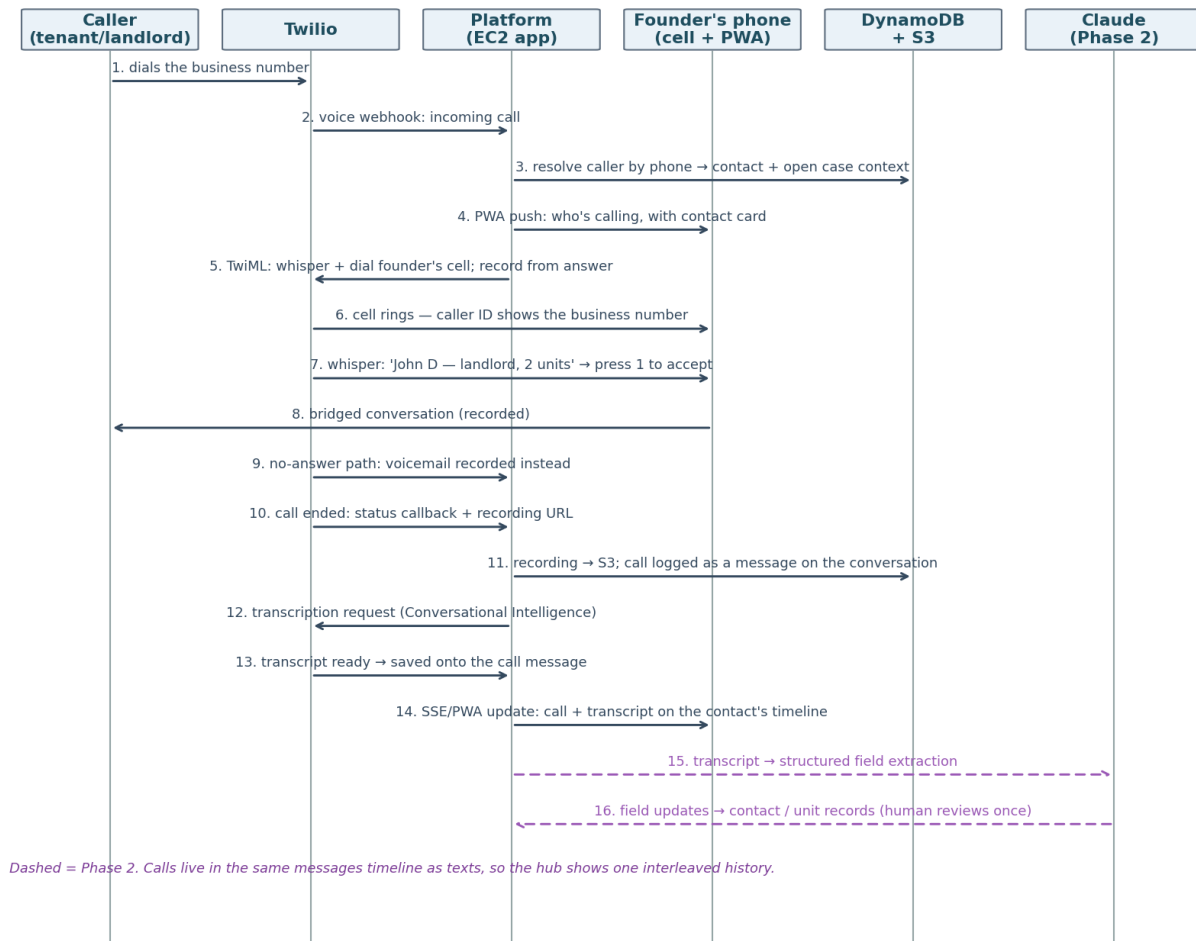


Figure 6 — Inbound call sequence: context push, whisper, bridge, recording, transcription, and logging as a message; dashed = Phase 2 extraction.

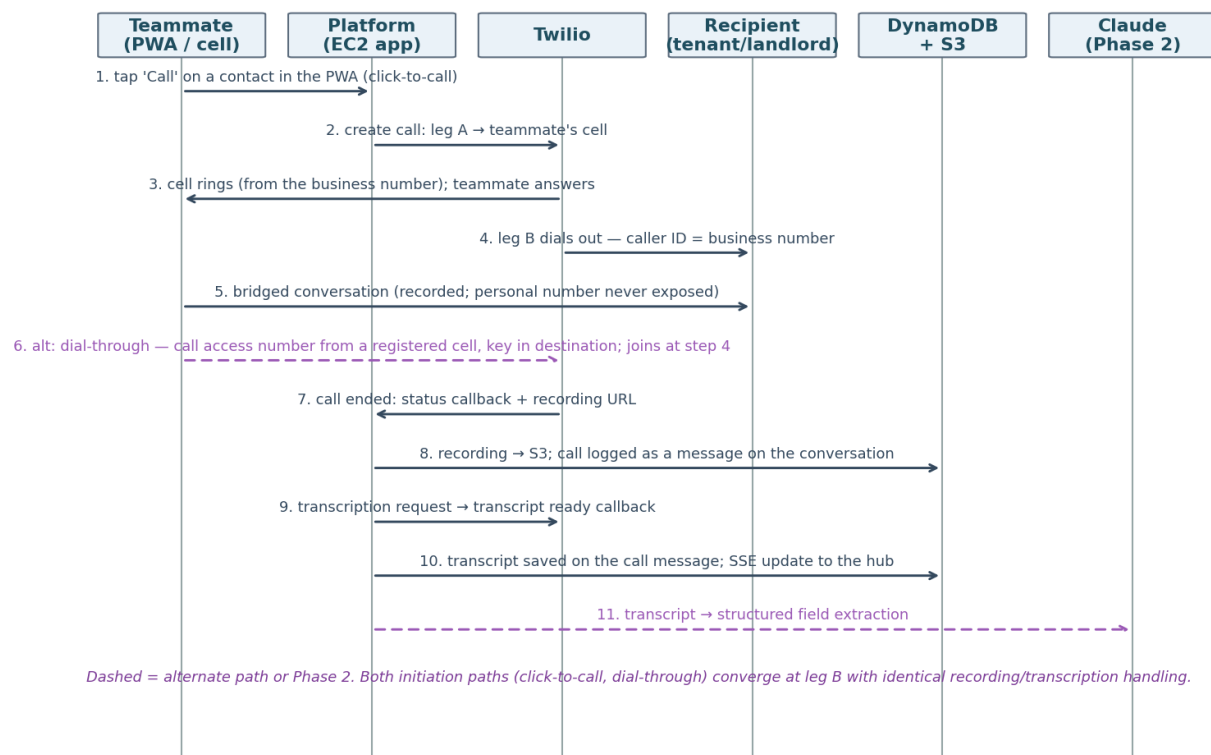


Figure 7 — Outbound call sequence: click-to-call (or dial-through) with business caller ID; both paths converge into the same recording → transcription → timeline tail.

7.2 The Rest

Integration	How it's used	Notes / cost
DocuSign eSignature API	One-click onboarding packet (v2 F1.2): templated welcome email + contract sent when lead_status flips to interested; DocuSign Connect webhook flips contract_status to signed automatically — no one checks an inbox. (Phase 3.)	Starter \$50/mo, 40 envelopes. Standard email signing — embedded signing needs a pricier plan and isn't required.
Igloohome API	Tour confirmed + ID verified → generate time-bound PIN for that property's lockbox; PIN texted in real time; issuance audited.	\$2 per active lock/month, only in months used. Offline algorithmic PINs — no connectivity needed at the door. Per-property tour_process decides whether this path or landlord/PM coordination applies. Phase 3 at earliest — whether lockboxes are in scope at all is under discussion (review).
Stripe Invoicing	Invoice auto-generates the moment rent determination lands (v2 F6.5/F7.1) and is due at move-in; ACH-first;	Placement fees run \$1,400–\$3,500 (v2). ACH ≈ \$5.60–\$7 all-in per invoice (0.4% capped \$2 +

Integration	How it's used	Notes / cost
	webhook reconciles payment into the revenue tracker. (Phase 4.)	0.8% capped \$5); cards ~3% — push ACH.
Amazon SES	Outbound welcome/onboarding emails; inbound hello@ mailbox for RTA submissions — SES receiving → S3 → AI confirms actual receipt, closing the 'I sent it / we never got it' gap (v2 F5.4). (Phase 3.)	\$0.10/1,000 outbound; receiving \$0.10/1,000. Domain verification + DKIM via Terraform.
AffordableHousing.com	Landlord prospecting source.	No public API or export — prospecting stays manual; cold-call transcripts enter via the intake flow instead.

8. Infrastructure as Code & Delivery

Terraform (≥ 1.15 , AWS provider v6) owns every resource — network, CloudFront, EC2, DynamoDB, S3, SES, Parameter Store, schedules, alarms, dashboards, IAM. State lives in S3 with native lockfile locking (`use_lockfile = true` — the DynamoDB lock-table pattern is deprecated). Two environments, dev and prod, in a single AWS account as fully separate stacks — every resource name-prefixed per environment, separate state files, separate IAM roles scoped per stack so dev credentials cannot touch prod resources. Drift control is manual for now (per review): console access stays read-only, every change goes through `npm run plan / npm run apply` wrappers from the dev machine, and the drift check is simply running the plan script before changes — scheduled drift automation comes later.

Delivery flow (npm-first, nothing automated — per review): `npm run test` (lint + unit/integration against DynamoDB Local) → `npm run deploy:dev` (build ARM64 image → push to ECR → SSM Run Command pulls, restarts compose, health-checks) → verify → `npm run deploy:prod` with the same image tag. Rollback is redeploying the previous tag (~2 minutes). The EC2 instance is rebuildable from scratch by Terraform + user-data bootstrap. CI/CD — tests on push, plan-on-PR, automated deploys — is deliberately deferred until the team grows (§14).

8.1 Local Development

`npm-first` (per review): `npm run dev` runs the app and worker natively with instant reload and a real debugger, auto-starting DynamoDB Local — the one local container — while the dashboard runs on Vite. `npm run dev:docker` exercises the production Dockerfile when container parity needs checking. Cloud-only dependencies are handled pragmatically: Claude calls hit the Anthropic API (or a recorded-response stub for offline work and tests); Twilio webhooks reach the laptop through a cloudflared/ngrok tunnel pointed at a Twilio dev number, with the messaging adapter's console driver (logs instead of sends) for pure-local flows; SES/Bedrock use the dev stack's resources when needed. A single `.env.example` documents every variable, including `LLM_PROVIDER` and `MESSAGING_PROVIDER`.

9. Logging & Observability

Designed for your stated need: when something goes wrong in the cloud, the answer must be in the logs. Every log line is structured JSON (pino) with correlation IDs — `requestId`, `conversationId`, `tenantId/landlordId`, `matchId` — threaded through every layer, so one Logs Insights query reconstructs any placement end to end. Containers log to stdout; Docker's `awslogs` driver ships to per-container CloudWatch log groups (90-day retention; audit trail lives in the `audit_events` table indefinitely). OpenTelemetry (not the end-of-life X-Ray SDK) traces requests and external calls into CloudWatch Application Signals.

Context across async boundaries — the envelope pattern. `AsyncLocalStorage` carries correlation IDs only within one process's execution flow; it dies at every async ingress (EventBridge Scheduler jobs, DynamoDB stream events, provider callbacks), which would leave worker logs orphaned from their originating request. The rule, enforced structurally: context rides the payload, and nothing touches the transport directly. All dispatch goes through a single `jobs.enqueue()` wrapper that stamps the current `correlationContext` (`originRequestId`, `conversationId`, `tenantId/casId`, W3C traceparent, hop count) into every job envelope — and, for one-off schedules, sets `ActionAfterCompletion: DELETE` so fired schedules clean themselves up rather than lingering as dead resources (§4.3); all intake goes through a single `defineJobHandler()` wrapper that extracts the envelope, mints a fresh `jobRunId`, and re-hydrates `AsyncLocalStorage` before any business logic runs — handlers never see raw events, so authors cannot forget what they never touch. Chained jobs inherit lineage automatically (enqueue inside a rehydrated handler reads the restored store). Jobs born without a request (weekly cadences) mint a root context with `originType: scheduler`;

payloads arriving without an envelope get a synthesized context plus a warning — never blind, never crashed. Twilio status and transcription callbacks cannot carry the envelope, so their middleware recovers context by lookup (MessageSid → message → conversation) before running identically. Carrying traceparent extends OpenTelemetry across the same boundaries, so Application Signals shows webhook and deferred job as one distributed trace. Enforcement is observable: a metric filter counts log lines missing a correlation ID, and a non-zero 'orphan logs' alarm means someone bypassed the wrappers.

What gets logged, exhaustively: every HTTP request/response (status, latency); every webhook received (verified/rejected, processing outcome); every outbound message with provider message ID and delivery-status callbacks; every AI decision — mode, action taken, handoff reason, guardrail hits, model, provider, tokens, latency; every matching run — counts, scores, duration; every job execution; every config/mode change with actor; and — per review, first-class — application errors: uncaught exceptions, unhandled promise rejections, and handled errors, each with full stack trace and correlation IDs, so code bugs surface as loudly as integration failures. Message bodies live in DynamoDB rather than logs (PII hygiene) — logs carry IDs that join to them.

Alarm	Fires when	Why it matters
Webhook failures	Signature rejections or 5xx on /webhooks/* exceed threshold	Inbound texts are the product's heartbeat — silent webhook failure = invisible lost conversations.
Send failures / delivery errors	Twilio error/throttling (429/30022) or undelivered-status rate spikes; outbound queue depth grows	Carrier filtering, A2P issues, or TPS budget breaches throttle the core loop; catch within minutes.
Runaway sends (echo loop)	Automated outbound to one conversation exceeds the per-minute breaker	Webhook echo chambers or bot-vs-bot loops (§7.1) — breaker trips, conversation flips to Manual, alarm pages.
Application error rate	ERROR-level structured-log events exceed threshold (metric filter)	Review: not just integration errors — code bugs surface immediately with stack traces and correlation IDs.
AI errors / handoff spike (Phase 2)	LLM call failures, or handoff rate deviates sharply	Model/provider outage or a prompt regression in production.
Escalation queue age	Oldest unhandled escalation or suggested reply exceeds SLA (e.g., 15 min business hours)	Manual-mode threads and guardrail handoffs must reach a human fast; protects tenant experience.
Stalled case / ladder	Application rung or RTA stage stuck beyond its threshold	v2's 'stuck > X days' requirement — voucher expirations and the 48-hour RTA window are revenue-critical deadlines.
Instance & containers	EC2 status-check fail, container restart loop, disk >80%	Single-instance topology makes host health a first-class alarm; auto-recovery handles the hardware case.
Job/DLQ depth	Scheduled jobs failing or dead-letter queue non-empty	Reminders and follow-ups are revenue-critical (voucher expirations, RTA 48h windows).
Drift (manual check for now)	npm run plan shows a diff when run before changes	Your zero-drift requirement — enforced by habit until the scheduled check is added (§14).
Orphan logs	Log lines missing a correlation ID exceed zero	Someone bypassed the context-envelope wrappers (§9) — correlation loss pages us instead of silently fragmenting traces.

A CloudWatch dashboard surfaces the funnel (messages in/out, active conversations by mode, suggestions/escalations pending, matches by stage, tours today, approvals in flight) next to system health. Alarms fan out via SNS to email/SMS.

10. Security, Privacy & Compliance

Platform security. EC2 in a public subnet, security group admitting only CloudFront's origin-facing prefix list (with a secret origin header verified in the app) and SSM — no SSH; the public IP exists solely for outbound API calls (review: no NAT gateway, no WAF for now — both are clean later additions); webhook authenticity by HMAC signature verification. Least-privilege IAM instance role scoped to exactly the tables, buckets, and APIs the app uses. Credentials as Parameter Store SecureStrings (review: no Secrets Manager), injected at runtime, never in the repo or image. Encryption at rest (DynamoDB, S3) and in transit everywhere. Team sign-in is direct Google OAuth restricted to the Workspace domain (review: no Cognito); sessions and RBAC live in the app — founder/admin sees everything including billing and landlord criteria; VA roles are scoped to conversations, tours, and applications per §12 of the Product Framework. Auth is a thin middleware, so when external users arrive (landlords, housing authorities — no Workspace logins), swapping in Cognito with Google federation is a days-scale change (§14).

Compliance posture. Fair housing: the guardrail layer routes any protected-class-adjacent topic to a human, the regression suite red-teams this continuously, and the `audit_events` table preserves who/what/why for every AI action. TCPA/A2P: texting only with recorded opt-in, automatic STOP/HELP handling, quiet-hours enforcement, and proper 10DLC registration. PII: voucher and screening data is sensitive — access scoping, message bodies kept out of logs, DynamoDB point-in-time recovery (35-day window, on from day one — review reversal) plus S3 versioning for media recovery, and data minimization in Claude prompts (only the fields the conversation needs).

11. Build Plan & Effort Estimate

11.1 Assumptions

- AI coding agents (e.g., Claude Code) write the large majority of code, tests, Terraform, and docs from a clean slate; estimates are calibrated to agent-speed building, not hand-coding.
- One technical lead directs agents, reviews and merges code, owns AWS/vendor accounts, and makes integration calls — roughly half- to three-quarter-time. The founder is available for domain rules (matching criteria, scripts, edge cases) a few hours weekly; codifying her instincts is a genuine dependency, not a formality.
- 'Working days' below are agent-driven build days including review and testing. Workstreams within a phase parallelize across multiple agent sessions, which is why phase calendar time is shorter than the day-sum.
- External lead times run concurrently from week 1: A2P 10DLC registration (days–3 weeks, gates production texting), a group-texting smoke test on the new Twilio account (GA since Oct 2024 — belt-and-suspenders only), Bedrock model-access request, DocuSign and Stripe account setup.
- Estimates include integration debugging and hardening but exclude visual design exploration, data migration from the Airtable/n8n MVP (parked, §14), and operational rollout (VA training, landlord re-papering).

11.2 Phase 0 — Foundations (1–1.5 weeks)

Workstream	Builds	Days
Repo & scaffolding	Monorepo, Express app + worker skeleton, dashboard shell, lint/test harness, Dockerfiles	2–3
Terraform baseline	VPC (public subnet), EC2 t4g + bootstrap, CloudFront/ACM, DynamoDB (PITR on), S3, ECR, SES, Parameter Store, budgets/alarms — one AWS account, separate dev/prod stacks	2–4
npm tooling	npm run dev / test / deploy / plan / apply scripts — no CI/CD per review	1–2
Local dev & logging core	npm-first dev env (DynamoDB Local container), seeds, pino + correlation IDs + error capture, async-boundary context envelopes (enqueue/handler wrappers, §9), OTel wiring, log groups/dashboards	2–3

11.3 Phase 1 — Feature Parity on the Platform (4–7 weeks)

Re-scoped per founder review: Phase 1 reaches feature parity with how the business runs today — every text and call to tenants and landlords happens manually through the platform (group texts included) and everything is tracked. Exit criterion: contacts auto-captured; tenant/landlord/unit records maintained with per-property processes; properties shared by filtered broadcast; tours and placements recorded on boards; business calls ring on and originate from the founder's cell through the platform number, recorded and transcribed; Quo fully retired. No AI in the loop yet — the platform is the system of record.

Workstream	Builds	Days
Messaging integration	Twilio Conversations adapter (1:1) + masked relay group threads with pool numbers (§7.1), signed webhooks with echo suppression + MessageSid idempotency, media → S3, delivery callbacks, opt-in/opt-out logging, manual sends from the hub	4–7
Voice bridging	Inbound ring to founder's cell (whisper, voicemail), click-to-call + dial-through outbound with business caller ID, recordings + transcripts saved to the record	2–4
Conversation hub (backend)	Conversations/messages model (call transcripts attached), assignment, SSE live updates	3–6
Conversation hub (UI)	Inbox, thread view, group-text support, per-message delivery states + failure badges with retry/call-instead actions (§7.1), profile side panel; installable mobile-first PWA with push notifications	6–10
Records & intake	Tenant/landlord/unit CRUD incl. per-property tour/application processes, contact auto-capture ('First Last - N Bed'), housing-fair form → auto welcome text, property intake forms	4–7
Filtered share broadcast	MVP-parity Share Properties: bedroom-size + housing-authority filters, merge fields, flyer link, suppression/save-for-later; token-bucket throttle + backoff sized to the A2P brand tier (§7.1)	2–4
Boards (parity)	Placement calendar (placed / changed-mind), tour schedule view, RTA board lite — manually updated	3–5
Auth & roles	Google OAuth (Workspace-restricted), sessions, founder/VA RBAC	2–3
Cutover & hardening	A2P registration, number port from Quo, end-to-end rehearsal, runbook	3–5

11.4 Phase 2 — Matching Engine + AI Layer (4–7 weeks)

Workstream	Builds	Days
Normalization gate	Validation + geocoding (Location Service), single-housing-authority enforcement, match_ready flags (F3.1)	3–5
Matching engine	Two-question scoring (fit + approval likelihood), per-authority scoping with MATCH_WITH_PORTABILITY_REQUIRED status (port-eligibility check, receiving-authority payment standard, friction-discounted rank, day-one team alert, per-market config)	5–10
Share → react → refine	Per-tenant reasoned shares (F3.3), feedback parser → preference updates → re-match (F3.5), graceful exits (F3.4)	4–7
AI orchestrator	Auto/Manual modes, hard handoff triggers, fair-housing guardrails, one-tap suggested replies, golden-conversation evals, LLM adapter (3 providers)	6–12
Conversational prescreening	AI-run field-complete intake over text (F2.2)	3–5

Workstream	Builds	Days
Extraction pipeline	Call transcripts + threads → structured fields on tenants/landlords/units — the 'never typed twice' requirement	3–5
Tour automation	Same-day booking + reminder ladder (F4.1), per-property tour router (F4.2), no-show detection (F4.4)	4–7

11.5 Phase 3 — Automation Depth (4–6 weeks)

Workstream	Builds	Days
Application ladder	Post-tour feedback collector, per-property application runner, four-rung follow-up ladder with stall escalation (F5.1–5.3)	3–6
RTA retrieval	Guidance by text, hello@ inbound mailbox (SES) with AI receipt confirmation, 48-hour clock (F5.4)	3–5
Approval pipeline	Auto relay-thread formation (masked), stage-ladder board, per-case timers + stuck alerts (F6.1–6.2)	4–7
Approval cadence	Weekly tenant checklists, retention check-ins with sentiment routing, rent-determination intake (F6.3–6.5); guided HUD-reporting checklist ladder for portability cases	3–6
Landlord onboarding	One-click packet (welcome email + DocuSign webhook) and self-serve onboarding (F1.2)	4–7
Call logging	Cold-call logger, lead_status/decline-reason taxonomy, nurture re-touch, onboarding-call checklist (F1.1/F1.3)	3–5
VA work queue	Escalations + stalled ladders as tasks; placements-per-teammate metric (F8.2)	3–5
Browser softphone	In-dashboard calling via Voice JS SDK	3–6

11.6 Phase 4 — Scale, Invoicing + In-App Calling (3–6 weeks)

Workstream	Builds	Days
Invoicing & revenue	Auto-invoice on rent determination (Stripe ACH; the amount is already captured by Phase 3's rent-determination intake), revenue tracker + landlord partner ledger (F7.1–7.2)	3–6
Lockbox ID gate	Igloohome PIN flow, no-ID-no-code state machine (F4.3) — in-scope decision pending (review)	3–5
Call companion app	React Native + Twilio Voice RN SDK (one codebase, iOS + Android); CallKit/ConnectionService ringing, VoIP push, in-call context card, deep links into the PWA; app → cell-bridge → voicemail routing ladder; TestFlight + Play internal distribution	8–15
Multi-market	Market entity, per-authority payment standards (manual tables per review), number pools per market, market-scoped matching and dashboards	4–8
Housing-authority workflow	Atlanta Housing contract workflow, voucher-expiry rescue reporting	3–6

Workstream	Builds	Days
Recertification	Already-housed tenant tracking, recertification reminders and flows	3–5
Metrics	KPI dashboard: placements/month, first-text-to-move-in, funnel conversion, vouchers saved, per-teammate capacity, tenant sentiment	2–4

11.7 Summary & Costs

Phase	Calendar	Technical-lead hours
0 — Foundations	1–1.5 weeks	20–35
1 — Feature parity (manual comms, everything tracked; Quo retired)	4–7 weeks	100–200
2 — Matching engine + AI layer	4–7 weeks	100–190
3 — Automation depth	4–6 weeks	90–175
4 — Scale, invoicing + call companion app	3–6 weeks	85–185
Total	16–27 weeks (~4–6.5 months)	≈395–785

On the review comment that the earlier numbers felt overstated: a 'day' here is an agent-build day including code review, integration testing, and rework — not keyboard time — and the roadmap spans nine external integrations plus a live cutover. A full-time lead running parallel agent streams lands near the bottom of each range (total ≈ 11–16 weeks); part-time leadership and integration surprises price in the top. The re-phasing also changes what the calendar means: the team operates fully on the platform at week 5–8, so everything after improves a working system rather than delaying launch. The dominant schedule risks remain non-code: A2P registration and the number port, Bedrock access, and codifying the founder's matching rules.

Itemized for founder review. Every quantity below is an assumption to be corrected, not a fact: the baseline pictures roughly 50 active tenants, 5–15 placements in the approval window, ~3,000 outbound + 2,000 inbound SMS segments, ~600 outbound MMS, and ~700 call minutes per month. Rates are published vendor prices (June 2026, us-east-1).

Service	Qty / assumption	Rate	Monthly	Notes
AWS — EC2 prod	1× t4g.small, 24/7	\$0.0168/hr	\$12.26	On-demand; a savings plan later cuts ~40%.
AWS — EC2 dev	1× t4g.micro, ~8h weekdays	\$0.0084/hr	~\$1.50	Stopped outside dev hours; \$0 when idle.
AWS — EBS	2× 20 GB gp3	\$0.08/GB-mo	\$3.20	Root volumes.
AWS — CloudFront	well under 1 TB + 10M req	always-free tier	\$0	Free tier covers this scale entirely.
AWS — DynamoDB	~50k writes, 200k reads,	\$1.25/M wr · \$0.25/M rd ·	~\$1.50	On-demand; grows with message volume.

Service	Qty / assumption	Rate	Monthly	Notes
	~2 GB + PITR	\$0.25/GB · PITR \$0.20/GB		
AWS — S3	~15 GB media, +5 GB/mo	\$0.023/GB-mo	~\$0.50	MMS photos, recordings, RTA attachments.
AWS — CloudWatch	2–5 GB log ingest, ~10 alarms, 1 dashboard	\$0.50/GB · \$0.10/alarm	\$3–6	Maximal-logging posture drives ingest; 90-day retention.
AWS — SES	~1k out + 1k in	\$0.10 per 1k each way	\$0.20	Welcome emails + RTA inbox (P3).
AWS — misc	ECR ~2 GB, Route 53 zone, Scheduler, Param Store	—	~\$1.50	Scheduler invocations effectively free at this volume; Parameter Store standard tier \$0.
AWS subtotal			≈ \$24–27	
Twilio — main number	1	\$1.15/mo	\$1.15	
Twilio — relay pool numbers	5–15 active placements	\$1.15/mo each	\$6–17	One per placement in the approval window; 30-day quarantine before reuse.
Twilio — A2P campaign	1 (low-volume mixed)	\$1.50–2/mo	\$2	TCR recurring fee.
Twilio — SMS outbound	~3,000 segments	\$0.0079 + ~\$0.003 carrier	~\$33	Includes relay fan-out; multi-segment messages count per segment.
Twilio — SMS inbound	~2,000 segments	\$0.0079	~\$16	No carrier surcharge on most inbound.
Twilio — MMS outbound	~600	\$0.0215 + ~\$0.005 carrier	~\$16	Property photos dominate.
Twilio — MMS inbound	~300	\$0.0165	~\$5	IDs, landlord property photos.
Twilio — Conversations MAU	~250 active participants	first 200 free, then \$0.05	~\$2.50	Active = sent/received in month.
Twilio — voice outbound	~400 min	\$0.014/min	\$5.60	Click-to-call + dial-through legs.
Twilio — voice inbound	~300 min	\$0.0085/min	\$2.55	
Twilio — recording	~700 min	\$0.0025/min	\$1.75	Storage ~\$0.35/mo at this volume.
Twilio — transcription	~500 min	\$0.024/min	\$12	Founder knob: transcribe all calls vs. business calls only.

Service	Qty / assumption	Rate	Monthly	Notes
Twilio subtotal			≈ \$90–115	
Claude conversation AI	Phase 2+: ~5k inbound msgs	Sonnet-class \$3/\$15 per M tokens; ~3k in / 0.3k out per msg	\$0 in P1; ~\$45–90 from P2	Model class is the lever; Opus-class roughly 5×.
DocuSign Starter	Phase 3+: 1 plan	\$50/mo (annual billing)	\$0 until P3, then \$50	40 envelopes/yr; revisit if onboarding volume exceeds it.
Stripe Invoicing	Phase 4+: per placement	≈\$5.60–7 per ACH invoice	\$0 until P4; then revenue-linked (~\$30–70 at 5–10 placements)	0.4% capped \$2 + 0.8% capped \$5; scales with revenue, not time.
Igloohome	scope TBD	\$2/active lock-month	\$0 unless adopted	Only in months a lock is used.
Phase 1 total			≈ \$115–145/mo	AWS + Twilio only — no AI, no SaaS yet.
Full-build total			≈ \$210–290/mo + revenue-linked Stripe	Messaging and AI scale with volume; infrastructure stays nearly flat.

One-time / annual item	Qty	Rate	Cost	Notes
A2P brand registration	1	\$4 one-time	\$4	TCR; \$8 re-file if rejected.
A2P campaign vetting	1	\$15 one-time	\$15	Syniverse.
A2P brand vetting (optional)	0–1	\$40 one-time	\$0–40	Only if >2,000 msgs/day to T-Mobile is ever needed.
Quo number port-in	1	typically \$0	\$0	Twilio doesn't charge for US local port-ins; Quo may have an exit step.
Google Play developer	1 (Phase 4)	\$25 one-time	\$25	Companion app distribution.
Apple Developer Program	1 (Phase 4)	\$99/year	\$99/yr	Annual, not one-time — first real recurring SaaS-like fee.
AI agent usage (the build)	1–2 Claude Max seats × 4–6 months	\$100–200/seat/mo	\$400–2,400	Dominates this table; tracks build duration. API-metered usage is the alternative at similar spend.

One-time / annual item	Qty	Rate	Cost	Notes
One-time total			≈ \$450–2,600	Effectively: A2P fees are noise; the build's AI usage is the real one-time cost.

12. Risks & Mitigations

Risk	Impact	Mitigation
A2P 10DLC delays or carrier filtering	No production texting — the product is texting	Register week 1 (no carry-over from Quo); registered traffic + opt-in hygiene; delivery-rate alarms. The adapter keeps a provider swap cheap.
Relay-thread confusion or misrouting	A member misunderstands who sees a message, or a recycled pool number routes a stale reply to the wrong placement	Intro messages name everyone connected; every relayed message carries a sender prefix; threads close with a notice at case end; pool numbers quarantine 30 days before reuse, and replies to closed threads surface flagged in the hub.
Quo → Twilio transition disrupts current MVP	Operational gap during cutover (texting and voice)	Decision made (review): port the Quo number. Build + register A2P on a temporary number first, rehearse, then port in a scheduled cutover — Quo keeps working until the port lands.
AI sends something wrong in Auto mode	Trust/compliance damage	Manual-by-default with one-tap AI suggestions; Auto earned per message-type (v2); hard guardrail triggers; golden-conversation CI suite; full audit trail; per-conversation kill-switch.
Fair-housing exposure	Legal risk	Deterministic topic routing to humans, red-team tests in CI, audit log, periodic human review of Auto-mode transcripts.
Single EC2 instance fails	Downtime until recovery	Stateless instance + auto-recovery + terraform rebuild ≈ minutes-to-low-hours RTO; acceptable at this stage, ECS path documented if SLA tightens. PITR (on from day one) gives DynamoDB a 35-day restore path.
Matching quality disappoints	The core differentiator underdelivers	Treat founder rule-codification as a first-class workstream with her review of ranked outputs against real history; log scores + reasons for every match for tuning.
DynamoDB friction for new query patterns	Slower dashboard/reporting iteration	Document-style items + in-memory matching absorb most churn; OpenSearch/stream-index path documented before it's needed.
Per-property process sprawl	'Never standardized' becomes unbounded scope	Processes stored as structured-but-free-form documents executed by AI + human, not hardcoded; new patterns cost data entry, not code.
Estimate optimism on integrations	Schedule slip	Integration-heavy items carry the widest ranges; external lead times started week 1; phase gates allow re-scoping at each exit.

13. Open Questions

1. Where is DNS for housingchoice.com hosted today? Route 53 is optional — a CNAME from the current host to CloudFront suffices (review). If it's easy, moving the zone to Route 53 keeps it in Terraform.

2. Housing-authority matching rule — verification came back mixed and needs the founder's operational answer. Confirmed (HUD): a tenant's voucher has exactly one administering authority at a time. Not confirmed: that a unit serves only one authority — HUD portability lets an out-of-jurisdiction voucher lease the unit (receiving PHA bills or absorbs), and program vouchers (GHV, Step Up) overlay geography. The engine now handles ports structurally (MATCH_WITH_PORTABILITY_REQUIRED, §5) — the narrowed founder question: surface portability matches from day one, or leave the per-market flag off until in-jurisdiction supply thins? And does she handle program overlays (GHV, Step Up) today?
3. Lockbox / self-guided tours: in scope at all? (Review: needs discussion.) Igloohome remains the recommended vendor if yes; nothing else depends on this decision.
4. Listing-link flyer: v2 leaves the exact fields TBD ('only specific information, kept simple on purpose') — founder to specify what appears.

14. Parked for Later (Review Decisions)

Explicitly out of scope for this build, kept here so they aren't lost:

- MVP data migration (Airtable/Supabase records, Quo history) — review: ideas tab, not this scope; ballpark 3–6 days when wanted.
- External logins for landlords and housing authorities — likely via Cognito with Google federation so the team keeps Workspace sign-in; auth middleware is built to make this a days-scale swap.
- CI/CD automation: tests on push, terraform plan on PR, auto-deploys with approval gates; scheduled drift detection.
- S3 lifecycle/backup policies and AWS Backup automation beyond DynamoDB PITR (which is now on from day one).
- WAF at the edge and NAT-gateway/VPC-origin private-subnet hardening — clean additions if abuse or compliance demands.
- Telnyx fallback integration behind the messaging adapter.
- Reuse/review of the two dashboard MVPs' UI patterns (bare-bones + housingchoice-hq) as design references.

15. Key References

Selected sources verified during research (June 2026):

- [HousingChoice — Product Spec & Feature Flow v2 \(supersedes the Technical Process Map\)](#)
- [Quo API — message send spec \(single recipient, no MMS\)](#)
- [Twilio — Group Texting in Conversations \(docs\)](#)
- [Twilio changelog \(Oct 31, 2024\) — Group Texting GA in Conversations \(supersedes the Mar 2022 restriction\)](#)
- [Twilio — Conversation with Participants API \(single-request group creation\)](#)
- [Telnyx — Group Messaging \(openly available, 8 participants\)](#)
- [Twilio — Programmable Voice pricing \(calls, recording, transcription\)](#)
- [Telnyx — Group MMS endpoint \(fallback provider\)](#)
- [Amazon Pinpoint end of support, Oct 30 2026 \(migration notice\)](#)
- [Anthropic Claude on Amazon Bedrock — model availability & access](#)
- [Claude Platform on AWS \(GA May 2026\)](#)

- [CloudFront VPC origins \(parked private-subnet hardening pattern, §14\)](#)
- [AWS X-Ray SDK end-of-support → OpenTelemetry migration](#)
- [Terraform S3 backend — native state locking \(use_lockfile\)](#)
- [DynamoDB data modeling foundations \(single vs multi-table\)](#)
- [HUD — Housing Choice Voucher portability \(one administering PHA; formal port process\)](#)
- [Georgia DCA — HCV jurisdiction \(149 counties; ~10 incl. Fulton/Atlanta run their own PHAs\)](#)
- [Igloohome API \(algorithmic time-bound PINs\)](#)
- [DocuSign developer plans & pricing](#)
- [Stripe Invoicing pricing](#)
- [Twilio A2P 10DLC fees & registration](#)
- [Cal.com closed-source announcement \(Apr 2026\)](#)